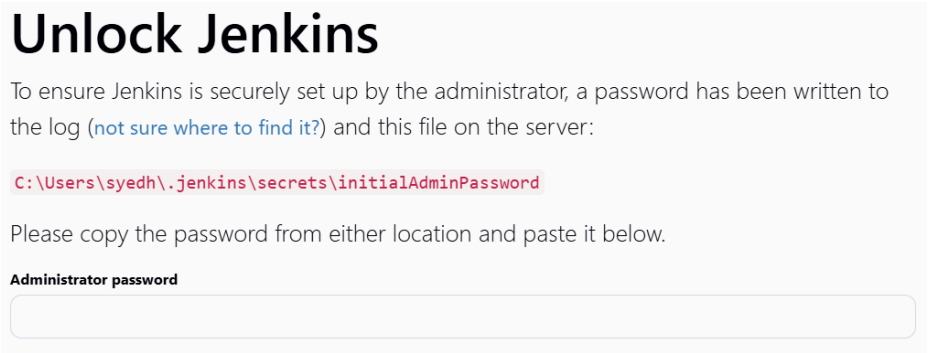
# Optimizing Build Pipeline with SAST

Stages:

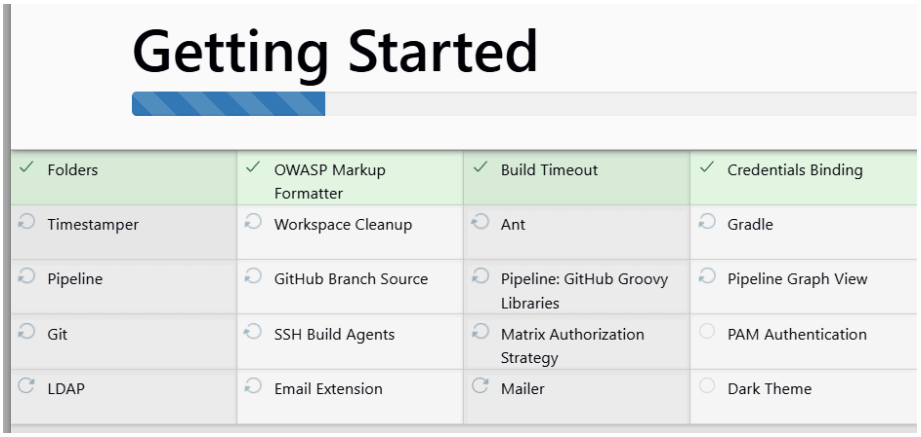
1. Setup Jenkins CI Server
2. Create a Build Pipeline (CI Build only)
3. Optimize the Build Pipeline with SAST

## Stage A - Setup Jenkins CI Server

1. Download JDK
    a. Browse link https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (windows)
    Browse link https://www.oracle.com/java/technologies/downloads/#jdk21-mac (mac)
    b. Login to SSO if required
    c. Extract the downloaded archive to C:\Oracle\Softwares folder
2. Download Jenkins
    a. Browse the WAR file link https://updates.jenkins.io/download/war/
    b. Download the latest WAR file using 'permalink to latest WAR'
    c. Copy the jenkins.war file to C:\Oracle\Softwares folder
3. Configure Jenkins
    a. Launch command prompt and execute below commands
        i. On windows platform perform the following
        ```
        set JAVA_HOME=C:\Oracle\Softwares\jdk-21.0.10
        %JAVA_HOME%\bin\java -jar C:\Oracle\Softwares\jenkins.war
        ```
        ii. On Mac machine perform the following
        ```
        export JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk-<FEATURE>.jdk/Contents/Home
        $JAVA_HOME/bin/java -jar jenkins.war
        ```
    b. Browse the URL http://localhost:8080 to get below page



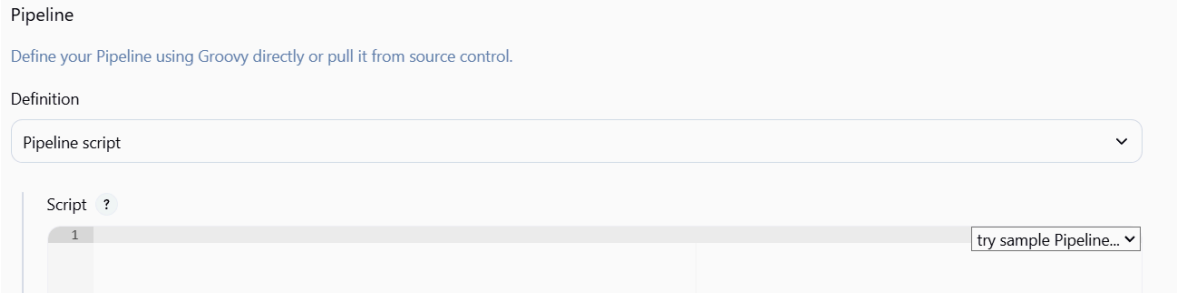

    c. Open the file named 'initialAdminPassword' to get the password
    Note that the path is mentioned on the browse page
    d. Provide the password → Click Continue
    e. Click on 'Install Suggested Plugins'
    f. Wait till the plugins are downloaded (Plugins are helpers of Jenkins)




    g. Click on 'Skip and continue as admin' over the 'Create First Admin User' screen.
    h. Click on Save and Finish

# Stage B - Create the CI Job (Build Pipeline) and execute it.

1. Browse the link ‘http://localhost:8080/’
2. Click on ‘New Item’
3. Provide the item name as ‘ci-pipeline’ → Choose ‘Pipeline’ as the item type → Click Ok
4. Scroll down to locate the ‘script’ section of pipeline




5. Copy and paste below script within the ‘Script’ textbox.
   Few Notes to consider:
   a. Please replace ‘bat’ with ‘sh’ in case you are using a Mac Machine.
   b. Please modify ‘your-user-name’ in the ‘REPO_URL’
   c. Please replace ‘..\\.venv\\Scripts’ with ‘../.venv/bin’ in case of Mac Machine.

```
pipeline {
    agent any

    environment {
        REPO_URL = 'https://github.com/your_userName/ai-devops.git'
    }

    stages {
        stage('Checkout') {
            steps {
                // Checkout the code from GitHub using the credentials
                git url: "${REPO_URL}", branch: 'main'  // Adjust branch if needed
            }
        }
        stage('Build') {
            steps {
                script {
                    // Execute Maven build commands
                    bat 'python -m venv .venv'
                    dir('sample-python-repo') {
                        bat '..\\.venv\\Scripts\\pip install -r requirements.txt'
                    }
                }
            }
        }
    }
    post {
        success {
            echo 'Build and test completed successfully.'
        }
        failure {
            echo 'There was an issue with the build or tests.'
        }
    }
}
```

6. Click Apply button → Click Save button
7. Click on ‘Build Now’ item on the left side of screen
8. Notice the Build History it should have #1 item → Click on #1 build
9. Click on ‘Console Output’ of #1 Build to see how the build executed
10. To update the existing CI job
    a. Browse http://localhost:8080
    b. Click on the build job ‘tddworkshop-pipeline’
    c. Click Configure

# Stage C - Optimize the Build Pipeline with SAST

1. Ensure the Ollama is running (i.e. the 'ollama serve' is running)
2. Commit and push the sast-scan.py agent code to GIT hub repository
```
cd c:\labs\ai-devops\sample-python-repo
git add sast-scan.py
git commit -m "Adding SAST Scan Agent"
git push -U orgin main
```
3. Update the pipeline script to add 'SAST Scan' stage
```
        stage('SAST Scan') {
            steps {
                script {
                    // Execute Maven build commands
                    dir('sample-python-repo') {
                        bat '..\\.venv\\Scripts\\python sast-scan.py'
                    }
                }
            }
        }
```
Updated pipeline looks as below

Script ?
```
8   stages {
9       stage('Checkout') {...}
15      stage('Build') {...}
26      stage('SAST Scan') {
27          steps {
28              script {
29                  // Execute Maven build commands
30                  dir('sample-python-repo') {
31                      bat '..\\.venv\\Scripts\\python sast-scan.py'
32                  }
33              }
34          }
35      }
36  }
37  post {
```
Click Apply → Click Save
4. Run the CI build process. Notice the build failing. Upon inspecting from job logs you can see SAST Scan agent generated failures and returned 'exit(1)' causing the build to fail.